



Informe: Explorando la maleabilidad en AES-CBC (y contraste con ECB)

Criptografía

Autores:

Luisa Fernanda Guzmán Santoya

Maria Isabel Solá Valle

NRC: 1984

Profesor:

Ricardo Luis Villanueva Polanco

Ingeniería de Sistemas y Computación

Universidad del Norte

27 de Septiembre de 2025

1. Introducción

En este informe presentamos nuestra experiencia implementando un ataque de *Padding Oracle* sobre el cifrado AES en modo CBC. Donde buscamos explicar de manera sencilla cómo funciona la idea, qué resultados obtuvimos al probar el código y qué recomendaciones se pueden aplicar para evitar esta vulnerabilidad.

2. ¿En qué consiste el ataque?

La idea básica es que, cuando se usa CBC (Cipher Block Chaining), cada bloque de texto plano M_i se obtiene con la fórmula:

$$M_i = D_k(C_i) \oplus C_{i-1}$$

donde D_k es el descifrado con la clave secreta, C_i es el bloque actual y C_{i-1} el bloque anterior.

El problema aparece cuando se usa un *padding* llamado PKCS#7 y el sistema **responde distinto** si el relleno es válido o inválido. Esa respuesta (un simple **True/False**) ya le da al atacante una pista para ir reconstruyendo el mensaje byte por byte.

2.1. Invariante de sufijo (explicado simple)

El truco es ir de atrás hacia adelante:

- Para el último byte, se busca un valor que haga que el padding luzca como 01.
- Para los dos últimos bytes, se ajusta para que el final parezca 02 02.
- Y así sucesivamente hasta completar todo el bloque.

Esto se llama “invariante de sufijo” porque se garantiza que el final del bloque siempre se ve como un padding válido.

2.2. Procedimiento general

El ataque se hace así:

1. Se modifica el bloque anterior (C_{i-1}) para forzar distintas combinaciones.
2. Se prueban todos los valores posibles (0-255) en el byte activo.
3. Cuando el servidor responde que el padding es válido, se deduce el valor intermedio y con él el byte real del mensaje.
4. Se repite el proceso de derecha a izquierda hasta obtener el bloque completo.

3. Resultados y Métricas

Para probar el código, usamos el mensaje: “**Amo el Helado de vainilla**” (25 bytes). Al ejecutarlo, obtuvimos lo siguiente:

- Total de consultas al oracle: **8448**
- Bytes recuperados: **25**
- Consultas por byte: $\frac{8448}{25} \approx 337,9$
- Consultas por bloque: $\frac{8448}{2} = 4224$

Escenario	Consultas por Byte	Consultas por Bloque	Total por Mensaje
Peor Caso Teórico	256	4,096	8,192
Valor Esperado Teórico	128	2,048	4,096
Implementación Real	338	4,224	8,448

Podemos notar que nuestra implementación consumió un poco más que el valor esperado (128 por byte $\rightarrow$ 2048 por bloque). Esto se debe a que nuestro código hace verificaciones adicionales (backtracking) cuando aparecen varias respuestas “válidas” en el padding, lo cual aumenta el número de intentos.

4. Contramedidas

Para evitar este ataque, se recomiendan varias medidas prácticas:

- Validar primero la integridad del mensaje con un MAC/HMAC, y solo después descifrar.
- Usar modos de cifrado autenticados como AES-GCM o AES-CCM, que no necesitan padding.
- Evitar dar mensajes de error distintos entre padding válido e inválido.
- Mantener tiempos de respuesta constantes y limitar el número de intentos por cliente.

5. Conclusiones

Este laboratorio nos permitió entender cómo algo tan pequeño como un mensaje de error puede filtrar suficiente información para romper la seguridad de un sistema. Aunque al principio el tema parecía complejo, al implementarlo paso a paso vimos que con lógica y paciencia se puede reconstruir un mensaje completo a partir de respuestas binarias. El aprendizaje más importante es que la criptografía no solo depende de algoritmos fuertes, sino también de cómo se implementan y de los detalles que a veces parecen menores.